

樹搜尋原型可行性報告 —— 最終版:v1 → v2 → v3 全弧 + 規模曲線 (Phase F-3)

產生方式:數字逐字取自 `tree_search/harness.py` (v1)、`tree_search/harness_v2.py` (v2)、`tree_search/harness_v3.py` (v3)、**15 份樹狀態檔案**(`competitions/playground-series-{s3e9,s3e14,s3e5,s3e3,s3e7,s3e1,s3e19,s3e11,s3e16,s3e20}/experiments_tree*.json`,涵蓋 `experiments_tree.json` / `experiments_tree_v2.json` / `experiments_tree_v3.json` / `experiments_tree_scale.json` 四種檔名)、`docs/scaling_experiment.md` (Phase E-5),以及對應 `STATUS.md` 的樹搜尋附錄;抽取腳本 `docs/scripts/build_tree_facts.py` → `docs/tree_facts.json`。

版本範圍:本報告涵蓋 3 個 harness 世代(v1/v2/v3)+ 1 個規模擴充實驗,共 **15 次樹搜尋執行、覆蓋 10 場競賽**(任務簡報原估 12 次,經 `find competitions -iname "experiments_tree*.json"` 精確盤點後為 15 次——3 v1 + 9 v2 + 1 scale + 2 v3,詳見下方完整結果總表)。

計畫書脈絡:ERA(Aygün et al. 2026, Nature)——用候選樹搜尋取代線性單路徑迭代,核心機制為節點評分、選擇規則、plateau 偵測與回溯、以及「想法注入」持續擴充候選集。本報告是計畫書 Stage 4(週 6-7)先遭驗證的最終調整:v1(Phase C-2/C-3,3 場)先驗證機制成立,v2(Phase D-2..D-6 + E-1..E-4,9 場)加入 ensemble-default 節點空間等四項升級並掃描剩餘全部競賽,E-5(1 場)量測節點預算的報酬曲線,v3(Phase F-1/F-2,2 場驗證跑)把掃描中發現的六項教訓收斂為 harness 預設行為。問題從「值不值得做」(v1)升級為「在什麼條件下、用多少評估數就能贏」(v2)再升級為「贏的機制能否系統化、且經得起自動化實戰驗證」(v3)。

1. 目的與設計

設計目標

`tree_search/harness.py` 是一個與競賽無關的通用引擎;每場競賽只需提供一個 `evaluate(config) -> score` 函式。設計選擇如下:

- **node = 一個完整、已評分(或已失敗)的解:** `{id, parent_id, mutation, config, score, status, wall_s}`。不是「部分想法」或「一個超參數變動」的抽象節點,而是一個可獨立重現、有 CV 分數的完整 pipeline 設定——這是刻意的簡化,換取「隨時可比較任兩個節點」的簡單性。
- **選擇規則(selection rule):**任一時刻恰有一條「活躍 lineage」(root 的某個第一代子節點所代表的候選子樹)。`select_next_parent()` 永遠在活躍 lineage 內,挑選「分數最好、且尚未達到 `MAX_CHILDREN_PER_NODE (=3)`個子節點」的節點來擴展。
- **plateau/backtrack:**每當活躍 lineage 新增一個子節點,若其分數未能刷新「新增前的全域最佳分數」,該 lineage 的未改進計數 +1;連續達到 `PLATEAU_STREAK (=3)`次,該 lineage 標記為「已 plateau」並從候選中剔除, `select_next_parent()` 改選次佳、尚未 plateau 的 lineage(即回溯)。兩個額外規則防止搜尋卡死:(1) 一條 lineage 若耗盡擴展預算也視為 plateau;(2) 若所有 lineage 同時 plateau,清空 plateau 旗標一次(reopen)。
- **solo + blend 雙節點空間(Phase C-2b 起,v2 起升級為預設):** solo (訓練一個模型,OOF/test 預測快取到磁碟)與 blend (對已快取的成員 OOF 做權重搜尋,不重訓)。這是修正 s3e9 教訓的關鍵設計:單模型節點空間永遠碰不到集成解法能到達的分數。
- **OOF 快取:**solo 節點的 OOF/test 矩陣存成 `tree_search/cache_<comp>/solo_<node_id>.npz`,後續 blend 節點直接讀取、不必重新訓練——這是 blend 節點能做到「近乎免費評分」的唯一原因(此假設本身在 v1 第 3 節即被 s3e5 推翻,見下)。

與 ERA 的對應與簡化

ERA(Aygün et al. 2026)理念	本原型的對應	簡化了什麼
候選樹取代單路徑迭代	node/lineage/select_next_parent 機制,完整實作	無
Plateau 偵測與回溯	PLATEAU_STREAK (=3)+ lineage 排除 + reopen-once 保底;v2 起 metric-aware 自適應(ADAPTIVE_PLATEAU_STREAK =5)	v1 固定閾值,v2 才依指標離散度動態調整
想法注入(idea injection)	v1:驅動腳本手寫固定佇列;v2 起 suggest_priors 對經驗庫關鍵字比對(簡化版)	仍非「執行期動態生成新方向」,只是把既有經驗結構化成可查詢清單
多機並行搜尋	單機序列, harness.py docstring 明載「no concurrent add_node calls」	無平行化
變異提案者可自我演化	固定為 agent 撰寫驅動腳本當下手動設計的規則式佇列	無運行時想法生成

簡言之:本原型完整驗證了 ERA 的**搜尋機制**,但 v1/v2 都尚未觸及 ERA 真正被引用的差異化能力——**想法注入**。v3 的六項規則(第 9 節)把 v1→v2 掃描期間人工發現的模式(邊界推進、後期突破需重開 blend lineage、探索性 burst)收斂為 harness 自身的預設行為,某種意義上是「把人類在 10 場掃描裡學到的候選生成直覺,提煉成系統的一部分」——仍不是 ERA 描述的執行期動態生成,但比 v2 的靜態經驗庫比對更接近。

2. 完整結果總表(15 次執行,10 場競賽,3 個 harness 世代)

#	comp	harness 版/Phase	對照最佳(線性 或前版樹)	樹最佳	判定	節點	關鍵招式
1	s3e9	v1 / C-2a	線性 12.07003	12.07459	負	20	(無;單模型空間結構性到不了 blend 天花板)
2	s3e14	v1 / C-2b	線性 340.59891	340.52635	勝	20	blend 節點型別 + FEAT 6 員 (多樣性成員)
3	s3e5	v1 / C-2c	線性 0.56769	0.56766	平(差 0.00003)	40	重新發現線性最適區,無新增益
4	s3e3	v2 / D-2	線性 0.838140	0.841442	勝	22	FEAT lineage tenure-prune(EDA 未驗證假設證實)
5	s3e7	v2 / D-3	線性 0.899893	0.900242	勝	22	prior-informed ensemble mechanics + dedup 活鎖修正
6	s3e1	v2 / D-4	線性 0.557088	0.556329	勝	23(1 敗)	top-code 感知 clip 寫進 metric_fn 內部
7	s3e19	v2 / D-5	線性 10.01946	9.75707	勝	22	auto_scale 全域 ×1.02(修時序 OOF 系統性偏低)
8	s3e11	v2 / D-6	線性 0.295648	0.295280	勝	24	CatBoost depth 邊界推進 10→12

#	comp	harness 版/Phase	對照最佳(線性 或前版樹)	樹最佳	判定	節點	關鍵招式
9	s3e9	v2 / E-1(復仇戰)	v1 樹 12.07459 + 線性 12.070034(全精度)	12.070034	v1 樹勝 (+0.004556)/ 線性 精確平	26	ensemble-default 節點空間 + 歷史種子復現 + coord-descent 精修
10	s3e5	v2 / E-2(復仇戰)	v1 樹 0.56766 + 線性 0.56769	0.57066	雙贏 (+0.00300 / +0.00297)	23(1 敗)	邊界推進 blend 成員 (LGBBOUND)+ 足額 k=800 權重搜尋預算
11	s3e16	v2 / E-3(酸性測試)	線性 1.33812(唯一有 真實 Kaggle LB 錨點)	1.33563	勝(-0.186% 相對)	27(3 敗)	raw/rounded 反轉鐵證 + FEATPRUNE + k=800 精修
12	s3e20	v2 / E-4(結構主宰)	Phase-B 基準 21.1487	21.0332(blend, 低信度)/ 21.0589(純結構)	勝	38	JOINT 多軸聯合移動 + YEARWEIGHTS 新軸
13	s3e3	v2-scale / E-5(規模曲線)	v2 樹 0.841442 + 線性 0.838140	0.845051	雙贏	80(95, 含 15 死路佔位)	explore-burst KITCHENBLEND(dirichlet k=800 全池混合)
14	s3e7	v3 / F-2(驗證跑)	v2 樹 0.900242 + 線性 0.899893	0.900455	雙贏	60(63)	explore burst mega- blend(prob→rank 空間交換)
15	s3e14	v3 / F-2(驗證跑)	v1 樹 340.52635 + 線性 340.59891	340.35572	雙贏	60(62)	explore burst mega- blend(34 員,28 員留有實質權重)

15/15 執行中,12 次明確勝過其對照組、2 次精確追平(#9 對線性、#3 對線性)、1 次明確負(#1,結構性節點空間問題已於 #9 用 v2 翻盤)。逐版與逐場分析見第 3-9 節;十場全覆蓋的彙整判定(僅取每場 v2/v3 最佳結果對線性,見第 10 節)為 **9 勝 1 精確平、0 負**。

3. v1:三場結果與逐場分析(Phase C-2a/b/c)

s3e9(單模型空間困於噪音上限)

Root 為既有正則化單一 LGB(RMSE 12.11061,5.2s);5 條第一代 lineage 全部是**單模型**變異。20 個節點全部評估成功,總牆鐘 125.4s。全樹最佳分數是 CatBoost 種子節點本身(12.07459),4 次回溯都沒能找到更好的方向。

教訓:線性迭代真正的最佳分數(12.07003)來自 7-way seed-bagged blend,樹搜尋因為節點空間限定在單模型,結構上就到不了那裡。這不是搜尋機制失敗,是候選空間設計失敗——直接促成 s3e14 加入 blend 節點型別,也是 v2 把 ensemble-default 節點空間變成第一項升級的原始動機(第 9 節 E-1 顯示這個結構缺口用 v2 就能翻盤)。

s3e14(勝:blend 節點 <1s、fold-exact 驗證)

Root 沿用既有調校 LGB(342.02154,52.1s)。7 條第一代 lineage:6 條 solo + 1 條 BLEND。20 個節點、0 失敗,12 solo + 8 blend。BLEND 種子節點一評分就是全樹最佳,第 9 個評估(node #8, 340.59485)就**超車**了線性迭代耗費 7 次實驗才找到的

340.59891;最終於 node #11(340.52635)拉開差距。2 次回溯都發生在最佳分數已找到之後。

贏的原因:winning node(#11)的第 6 個成員——把 21 特徵集再砍 3 個溫度欄位的 LGB——線性迭代從未試過這個成員組合; 它的價值純粹是 blend 多樣性,只有節點空間能展開到 ensemble 才碰得到。

s3e5(平:成熟 blend 空間只被重新發現;離散指標階梯面)

Root 為 Optuna 直接以 post-rounder QWK 為目標調校出的 LGB(0.56244,1.4s)。8 條第一代 lineage。40 個節點、0 失敗。樹最佳是 node #11(4-way blend),QWK **0.56766**——與線性最佳 0.56769 只差 **0.00003**,量級小到單一樣本的切點歸屬就能翻轉。

離散指標帶來的三個新現象:(1) 分數面呈階梯狀,平手極常見(4 組 5 位小數完全相同);(2) PLATEAU_STREAK=3 對離散指標實質上更嚴格(8/8 lineage 全數 plateau,三場中最徹底);(3) 「blend 便宜」的假設不成立(每個候選權重都要做 Nelder-Mead 擬合,單一 blend 節點成本升到 ~45-46s,與 solo 節點同量級)——這三個發現直接催生 v2 的自適應 plateau 機制。

4. 樹搜尋何時贏——v1 三場歸納的邊界條件

1. 贏的條件:blend 組合空間存在,且尚未被線性迭代充分開採——s3e14 是 v1 唯一的勝場。
2. 平/負的條件:線性迭代已把該指標的可用信號榨乾,或節點空間設計本身不對——s3e5 是前者、s3e9 是後者。
3. 離散指標需要調整 plateau 規則——QWK-after-rounder 這類指標的分數面呈階梯狀,固定 PLATEAU_STREAK=3 比在連續指標上更快把每條 lineage 判定為停滯。
4. 「blend 便宜」是指標相關,不是普遍真理——任何以「blend 節點近乎免費」為前提的搜尋預算規劃,必須先確認該指標的後處理成本結構(v3 第 6 項成本護欄正是為此而生)。

5. 已知缺口(v1;四項已於 v2 修正/實作,詳見下節)

- 重複子節點:blend fallback 在 parent config 不變時會對同一 parent 重複產生同一組成員,浪費 plateau 額度。→ v2 以 `find_duplicate_config / add_node` 內建拒絕修正。
- blend-rounder 成本:離散指標(QWK)下 blend 節點成本追上甚至超過部分 solo 節點。→ v2 掃描證實可再推廣:blend 成本不只隨指標離散度變,也隨資料列數變(見第 6 節)。
- 無想法注入(ERA 第二支柱):三場的每條 lineage 變異佇列都是驅動腳本裡手寫的固定清單。→ v2 加入 `suggest_priors` (經驗庫關鍵字比對,無 LLM 呼叫)作為簡化版想法注入——仍非執行期動態生成新方向,見第 7 節誠實讀法。
- 單場單樹,無跨場遷移:三棵樹彼此獨立。→ v2 的 `suggest_priors` 部分解決,但候選集本身仍是每場手寫。

6. v2:四項升級與 D 掃描(5 場)+ E 掃描(4 場)——9 場全勝/翻盤

四項升級(`tree_search/harness_v2.py`)

1. **Ensemble-default 節點空間:** `kind ( solo / blend )` 升級為節點的第一級欄位。 `eval_blend(cache_dir, members, metric_fn, weight_search=...)` 對已快取成員做 Dirichlet 或 grid-simplex 權重搜尋。任何後處理規定必須寫在 `metric_fn` 內部。
2. **Metric-aware 自適應 plateau:** `tie_rate(tree)` 量測目前所有已評分節點中「完全同分」的比例; `tie_rate > TIE_RATE_THRESHOLD` (0.15)時停滯門檻從 `PLATEAU_STREAK (=3)`放寬為 `ADAPTIVE_PLATEAU_STREAK (=5)`。
3. **子節點去重:** `config_hash (sha256)+ find_duplicate_config, add_node` 預設對雜湊撞見既有節點的候選直接拒絕。

4. 經驗庫變異先驗: suggest_priors(comp_meta) 對 knowledge/experience.md 的每個標題做關鍵字比對,逐字回傳證據標註 bullet。

D 掃描(Phase D-2..D-6,5 場,首次全面掃描):5/5 全勝

s3e3(AUC,小樣本 1,677 列)、s3e7(AUC)、s3e1(RMSE,地理特徵)、s3e19(SMAPE,TimeSeriesSplit)、s3e11(RMSLE,36 萬列)——涵蓋 4 個指標家族、2 種 CV 方案、3 個資料規模量級,全部在 9-21 次評估內超越線性迭代最佳(完整數字見第 2 節總表 #4-8)。

E 掃描(Phase E-1..E-4,4 場,補完剩餘場次 + 兩場復仇戰)

- **E-1 s3e9 復仇戰:**v1 曾以 12.07459 輸給線性 12.07003(第 3 節)。v2 用 ensemble-default 節點空間,在第 14 個評估(BLEND 種子本身)就達到 **12.070034**——與線性迭代 7-way seed-bagged blend 的全精度分數逐位相同(**精確追平**),同時把 v1 的單模型天花板甩開 0.004556。搜尋機制自行發現了 seed-bagging + 加權混合這個線性迭代原本靠人工試出來的招式。
- **E-2 s3e5 復仇戰:**v1 曾以 0.56766 對線性 0.56769「統計上打平」。v2 用邊界推進(LGBBOUND,把 tuned-LGB 三個卡在 Optuna 搜尋盒邊界上的超參往外推)產生一個 solo 較弱(0.55784)但異質的 blend 成員,拿到 0.297 權重,把最終 blend 推到 **0.57066**——同時擊敗 v1(+0.00300)與線性(+0.00297),差距是 v1↔線性 gap(0.00003)的約 100 倍,不再是切點噪音量級。本場也是自適應 plateau 機制的首次實戰,結果是**全程休眠**(tie_rate 恆為 0)——因為子節點去重已經修掉了 v1 那種 duplicate-children 造成的假性同分,machinery 之間有依賴關係:「dedup 修好後,tie-neutrality 的觸發條件反而變得罕見」。
- **E-3 s3e16 酸性測試:**Phase B 的線性迭代在這場已踩到「取整 MAE」的已知失敗模式(raw OOF 單調改善,rounded OOF 反而變差兩輪)。v2 用 k=800+coordinate-ascent 權重搜尋直接對 rounded MAE 決策,找到 **1.33563**(-0.186% 相對於線性 1.33812)——且勝出組合是 raw MAE 1.35712(比線性冠軍的 1.35589 更差)但 rounded MAE 更好,與 Phase B 失敗模式完全鏡像對稱,是「決策必須基於 rounded 分數」這條規則最強的一次獨立確認。s3e16 也是 v2 全掃描**唯一有真實 Kaggle LB 錨點**的一場(Public 1.34356 / Private 1.34075,CV↔LB 差距 0.00544)。
- **E-4 s3e20(結構主宰地形):**此場已知純結構訊號(location-week 歷史均值)完勝所有 GBDT。樹搜尋在已被人工調到位的單軸(W2020、WNB)上誠實地「打平」——證明 Phase-B 手動調校本身沒有明顯單軸漏洞;真正的增益來自 (a) 一個全新結構軸 YEARWEIGHTS(異常年降權概念推廣到 2019/2021)與 (b) JOINT lineage 把 4 個結構旋鈕一步到位共同移動,複合拿到純結構最佳 21.0589(-0.425%)。最終疊加 GBDT 的邊際混合到 21.0332(-0.546%),但**這個混合權重只在 3 折 LOYO CV 上、直接對同一份 OOF 擬合**,信心低,視為 CV 噪音範圍內的邊際發現(見第 12 節誠實但書)。此場評估成本近乎零(38 節點 + 15 次回溯僅 33.3 秒),說明「結構主宰」與「模型主宰」地形不只解的形狀不同,搜尋預算的稀缺程度也天差地遠。

E 掃描收官

v2 在 D+E 兩輪掃描共測遍 9 場(D 的 5 場首測 + E 的 4 場:2 場翻盤/加強 v1 結果 + 2 場全新酸性測試),結果 **9 勝 0 負**(僅 s3e20 的 blend 疊加部分標記低信度,純結構主體仍穩健勝出)。加上 v1 的 1 勝 1 平 1 負,v1+v2 合計已覆蓋全部 10 場競賽中的 10 場(s3e9/s3e5 各有 v1+v2 兩次執行,詳見第 2 節總表)。

7. 先驗 vs 在地洞見

suggest_priors 的「先驗命中率」是系統性量化「想法注入值多少」的機會。D 掃描 5 場的 informed vs uninformed 勝率:s3e3 14.3% vs 14.3%(打平)、s3e7 62.5% vs 16.7%(先驗明顯優於)、s3e1 100% vs 62.5%、s3e19 33% vs 44.4%(先驗反而略輸)、s3e11 100% vs 18.2%。E 掃描 4 場延續同一模式:s3e9 v2 informed 36.4% vs uninformed 0%(小樣本)、s3e5 v2 33.3% vs 0%、s3e16 38.5% vs 0%、s3e20 informed 33.3% vs uninformed **42.9%**(prior 標註節點勝率反而略低於無 prior 節點)。

跨全部 9 場 D+E 反覆出現的發現:「先驗定下限,在地洞見定上限」。先驗(經驗庫)持續正確地「提名該試什麼方向」,價值主要是避免浪費算力在已知死路上。但每一場真正拉開分數差距的最大單一槓桿,始終來自該場自己的 EDA/comp-local 洞見或搜尋機制本身的結構性改變,而非經驗庫比對:s3e3 的 tenure-prune、s3e1 的 top-code clip、s3e19 的 auto_scale、s3e11 的 depth-boundary-push,以及 E 掃描新增的兩個範例——s3e5 v2 的 LGBBOUND(邊界推進本身,而非某條經驗庫 bullet 直接命中)、s3e20 的 YEARWEIGHTS/JOINT(全新結構軸 + 聯合移動,經驗庫裡沒有這兩條規則)。第 10 節的「boundary-push 3 場」與「burst 3/3」正是這個發現的兩個具體、可重複的機制化版本。

8. 誠實但書(v1+v2,9 場)

- **v1 三場**:s3e9 樹搜尋結構性到不了 blend 空間(已用 v2 翻盤);s3e5 的「平手」量級小到單一樣本切點歸屬就能翻轉;三棵樹之間無知識遷移。
- **D 掃描 5 場**:全部 OOF-only,未經 Kaggle LB 驗證;s3e19 有 fold-5 double-dip 疊加 OOF 擬合的雙重樂觀偏誤(詳見第 12 節);s3e7 掃描期間曾出現搜尋層級活鎖(已修正)。
- **E 掃描 4 場**:s3e16 是唯一有真實 LB 錨點的一場,其餘 8 場(含 v1)仍是 OOF-only;s3e20 的 GBDT-blend 疊加部分是 3 折 LOYO CV 上對同一份 OOF 直接擬合的邊際發現,信心低。

完整、彙整過的誠實但書清單(含 v3)見第 12 節。

9. v3:六項規則與 F-2 驗證跑

六項規則(`tree_search/harness_v3.py` ,把 v1→v2 掃描期間的人工發現收斂為 harness 預設行為)

1. **預算與停止策略**(`init_budget / update_phase / should_stop`):預設總預算 60 個已評估節點;相位機 exploit → explore_burst → stopped——所有已知 lineage 都 plateau 後強制進入 explore_burst(驅動腳本被期待注入 5–8 條新的長射程 lineage),burst 開始後若連續 20 次評估未刷新全域最佳即停止,任何情況下達到總預算硬停止。證據:E-5 的 80 節點曲線——3 次 explore-phase 改進全部來自強制 burst,且沒有停止規則時浪費了 28 個(占 80 節點預算 35%)閒置評估。
2. **去重消耗預算**(`add_node` 的 dedup 路徑):同一 parent 的候選連續兩次被 dedup 拒絕,即燒掉一個 `status="failed"` 佔位子節點,讓 parent 自然計滿 `MAX_CHILDREN_PER_NODE`。證據:E-5 建置時發現 v2 的 dedup 拒絕不消耗擴展額度,曾讓 kitchen-sink blend lineage 的 fallback 池耗盡後無限重複提案同一組態,把搜尋卡在 50/80 節點。
3. **post-plateau solo 突破自動重開 blend lineage**(`reopen_blend_lineage_on_solo_breakthrough`):當一個 solo 節點成為新的全域最佳,任何已 plateau 的 blend lineage 自動重開。證據:D-6(s3e11)的全樹最終最佳是 phase-1 BLEND lineage 已 plateau 之後才出現的 depth-12 CatBoost solo,原本需要人工開「phase 2」才拿到那個增益。
4. **邊界推進為一等公民變異型別**(`boundary_candidates`):自動標記任何落在其宣告搜尋空間邊緣(`edge_frac=0.05`)內的超參,並提出往外推的候選。證據:D-6/E-2/E-3 三場(s3e11 depth 10→12、s3e5 v2 的 LGBBOUND、s3e16 的 learning_rate)各自的單一最大槓桿都是這個模式(詳見第 10 節)。
5. **權重搜尋預設 k=800 + coordinate-ascent 精修**:k 預設從各呼叫端各自為政改為統一 800,並在粗搜之後跑一輪座標上升精修。證據:E-2/E-3 顯示粗網格會靜默地打平(多個候選權重向量四捨五入到同一個離散化分數),必須同時夠寬(k=800)且加精修才找得到真正的最優盆地。
6. **指標感知的 blend 成本護欄**(`eval_blend_with_cost_guard`):blend 評分若超過門檻(預設 45 秒)才自動粗化權重搜尋預算,且必留一條顯式警告紀錄,絕不靜默粗化。證據:C-2c(s3e5)的 QWK blend 節點成本 ~45–46 秒,追上 solo 節點量級;E-2 進一步顯示靜默粗化只會打平(丟失真訊號)而不會有人發現。

F-2 驗證跑(s3e7、s3e14,以 v3 預設自動策略端到端重跑)

驗證問題:harness_v3 的預設自動策略(相位機 + 自動停止 + 去重耗算 + 重開觸發 + 邊界推進 + k=800 精修 + 成本護欄)能否在真實跑動中端到端運作、不比 v2 退步,且相位機真的做了實質工作?

答案:兩場都是。且兩場超出 v2/v1 樹的全部增益,100% 來自相位機的強制 explore burst。

- **s3e7:**v2 曾以 0.900242 勝過線性 0.899893(D-3)。v3 exploit 階段在第 39 個評估耗盡所有 9 條第一代 lineage 的 plateau 額度,重現 v2 的 4-way SEEDBAG blend(0.900054,eval 11)為 exploit 天花板;explore burst 自動於 eval 39 觸發,注入 5 條長射程 solo(DART/extra-trees/深度CAT/lossguide-XGB/depth2-LGB,全部 solo 較差)+ 1 個 kitchen-sink mega-blend(38 員 rank-space Dirichlet+coordinate-ascent);mega-blend 拿到 **0.900455**,同時擊敗 v2(+0.000213)與線性(+0.000562)。在 60/60 硬頂停止,burst 後 13 次評估未再刷新最佳(< 20 次耐心值,數值上限先到)。
- **s3e14:**v1-proto 曾以 340.52635 勝過線性 340.59891(C-2b)。v3 在 eval 9 超車線性、eval 11 追平 v1-proto(晚 1 個評估,同一組成員),exploit 階段磨到 340.45150(eval 16)後 22 次評估無改善;explore burst 於 eval 38 觸發,34 員 kitchen-sink mega-blend(其中 28 員保留 >0.005 權重,不同於 s3e7 的 29/38 歸零——在這個雜訊更大的目標上,「廣度平均」本身就是訊號)把分數推到 **340.35572**,同時擊敗 v1-proto(0.17063)與線性(0.24319)。同樣在 60/60 硬頂停止,burst 後 16 次評估無改善。

邊界推進的 4 次確認:s3e7 的 `boundary_candidates()` 自動標記出 root 的 `max_depth` 卡在 Optuna 盒下界([3,12] 選中 3),自動生成的推進變異(depth 2)本身 solo 較差,但作為 burst mega-blend 成員拿到 0.114 權重——第 4 個確認「邊界值得推」的資料點,且這次是 harness 自動找到,不是人工重讀 Optuna trial 表。s3e14 的邊界檢查跑了但誠實地一無所獲(最近的超參離盒邊也有 ~13% log-space 距離)——這正是這個機制被期待的行為:檢查了,誠實地沒找到,而非被迫找出點什麼。

其他規則的實戰結果:去重耗算在 s3e7 觸發 3 次(crash-resume 期間重播的重複提案,全部正確燒掉佔位節點、零評估預算浪費);重開-on-breakthrough 在兩場都從未觸發(blend 從很早期就領先,沒有 solo 後來居上的情境);成本護欄在兩場都從未觸發(38 員 AUC blend 僅 11.6 秒 < 45 秒門檻)。

10. 十場全覆蓋圖景

以每場競賽「v2/v3 曾達到過的最佳結果」對線性迭代最佳分數彙整(見 `docs/tree_facts.json` 的 `ten_comp_coverage` 區塊,逐場取 min/max 後與 `linear_reference` 相減,無條件依指標方向判定):

10 場中 9 勝、1 精確平、0 負——唯一的「精確平」是 s3e9(v2 的 12.070034 與線性迭代全精度 12.070034 逐位相同),其餘 9 場(s3e14/s3e5/s3e3/s3e7/s3e1/s3e19/s3e11/s3e16/s3e20)全部是明確的勝。這比 v1 三場的「1 勝 1 平 1 負」是質的躍升——結構性缺口(s3e9 的單模型空間)被 v2 的 ensemble-default 節點空間徹底解決,原本平手的一場(s3e5)也被 v2 的邊界推進機制轉為明確勝場。

本次全 15 執行、10 場覆蓋歸納出三個可重複的機制發現:

1. **先驗定下限,在地洞見定上限**(第 7 節已詳述)——經驗庫命中持續避免浪費算力在已知死路,但真正拉開分數差距的槓桿(tenure-prune、top-code clip、auto_scale、depth-boundary-push、LGBBOUND、YEARWEIGHTS/JOINT)全部來自該場自己的 EDA 或搜尋機制的結構性改變。
2. **explore burst + kitchen-sink mega-blend:3/3**——所有三次「相位機強制注入探索性 burst」的場次(E-5 的 s3e3 scale、F-2 的 s3e7、F-2 的 s3e14),post-exploit-phase 的全部增益都來自 burst 本身注入的 kitchen-sink mega-blend,沒有一次是某個手寫長射程 solo lineage 單獨貢獻的。三場的增益分別為 +0.001527(s3e3 scale,對 exploit 天花板 0.843524)、+0.000401(s3e7,對 0.900054)、以及 s3e14 從 340.45150 降到 340.35572(改善 0.09579)。
3. **boundary-push:3 場**——s3e11(D-6,CatBoost max_depth 10→12,solo 0.295779→0.295461)、s3e5 v2(E-2,LGBBOUND 把 max_depth 從 3 推到 2,solo 分數是 0.55784,但拿到 0.297 blend 權重)、s3e16(E-3,learning_rate 從 Optuna 盒邊界 0.0102 推到 0.005,solo rounded MAE 1.33979→1.33950)——三場的單一最大槓桿都源自

「Optuna 最優解卡在搜尋空間邊界上」這個模式。s3e7 的 F-2 跑額外提供第 4 個確認資料點(邊界推進成員拿到 0.114 blend 權重),差別在於這次是 v3 的 `boundary_candidates()` 自動找到,而非人工重讀 Optuna trial 表——是「把人類直覺變成系統預設」這條 v3 設計原則最乾淨的一次證據。

11. 規模曲線章(Phase E-5, docs/scaling_experiment.md)

D-2 的 22 節點 s3e3 掃描找到 AUC 0.841442,但當時仍有可見的頭寸(四條 lineage 已 plateau,但搜尋在耗盡候選空間前就先碰到 22 節點上限)。E-5 把同一個搜尋 regime 延伸到 80 個已評估節點的預算,直接量測分數-評估數曲線,問題是:曲線在哪裡走平,後期回溯是否值得?

曲線走勢(逐字自 `experiments_tree_scale.json` 的 `curve` 欄位、由 `docs/scripts/build_tree_facts.py` 的 `curve_summary()` 機械抽取,與 `docs/scaling_experiment.md` 手工整理的同一份數字互相印證):全程 10 次全域最佳刷新,exploit 階段(eval 1-39)貢獻 7 次、explore 階段(eval 41 起)貢獻 3 次。Exploit 階段的最後一次改進在 eval 39(0.843524),此後 exploit 階段自身沒有再改進;explore burst 觸發後,3 次改進全部集中在 eval 45-52(KITCHENBLEND lineage),之後直到 eval 80(全跑結束)再無任何改進——idle tail 長達 28 個評估,占 80 節點預算的 35%。全跑實際牆鐘時間(含快取重用、失敗佔位節點)97.4 秒。

Stage-4 預算規則(`docs/scaling_experiment.md` 原文,經 `docs/tree_facts.json` 的 `v3_constants` 區塊確認已如實編碼進 `harness_v3.py` 的預設值):(1) 預設節點預算 60(exploit ~35-40 + 強制 explore burst 5-8 條長射程 lineage);(2) burst 後連續 15-20 次評估無改善即停(v3 預設 20);(3) 數值上限 60,不論相位機是否觸發都硬停。這三條規則正是 `harness_v3.py` 的 `DEFAULT_TOTAL_BUDGET=60`、`EXPLORE_BURST_MIN/MAX=5/8`、`DEFAULT_POST_BURST_PATIENCE=20` 常數的直接來源。

與全部 15 次執行的交叉校準(`docs/tree_facts.json` 的 `budget_efficiency` 區塊,對每次執行機械重算「最佳解出現在第幾次評估 / 總評估數」比值與尾端閒置評估數):15 次執行的 best/total 比值介於 0.10(s3e9 v1)到 1.00(s3e20,結構主宰地形評估成本近零、最後一個節點才刷新最佳)之間,均值約 0.65;idle tail 介於 0(s3e20)到 28(s3e5 v1 的 40 節點跑、以及 s3e3 scale 的 80 節點跑——同一個 28-評估閒置尾端在 2 倍的預算規模下重現,顯示這是 harness 本身的搜尋動態特徵,不只是特定預算大小下的巧合)之間。

12. 誠實但書(v1+v2+v3,全 15 執行)

- 全部 15 次執行,只有 s3e16(E-3)有真實 Kaggle LB 錨點**(Public 1.34356 / Private 1.34075,CV↔LB 差距 0.00544)——其餘 14 次的「樹最佳勝過對照組」結論都只在 OOF 分數上成立,尚未經 Public/Private Leaderboard 驗證。本次週末批次執行「不碰任何 token」是鐵則,OOF 勝出不保證 LB 勝出,尤其在權重搜尋/後處理參數本身就是對同一份 OOF 擬合出來的情況下。
- s3e19 的 fold-5 double-dip + OOF-擬合但書**:線性迭代原本的 10.01946 已帶有「fold 5 同時是 Optuna 調參目標、又是 5 折 OOF 的其中一折」的雙重使用樂觀偏誤;樹搜尋的 9.75707 在此之上再疊加 `auto_scale` ($\times 1.02$ 純量)與種子選擇這兩層直接對 114,000 列 OOF 擬合的參數。誠實的讀法是「實際 SMAPE 應顯著低於 10.02,但不應直接讀成 9.76」。
- s3e20 的 GBDT-blend 疊加部分信心偏低**:純結構節點(21.0589)是穩健結論,但疊加 GBDT 的最終 21.0332 只在 3 折 LOYO CV 上、直接對同一份 OOF 擬合出 2.17% 的極小權重,大機率是 CV 噪音而非真實訊號,與 Phase-B 殘差診斷的既有結論一致(感測器特徵對此目標不含結構之外的可預測訊號)。
- v3 的 auto-stop(耐心計數器)在兩場 F-2 實戰中都未曾真正觸發**——s3e7、s3e14 都是 explore burst 持續改善全域最佳,耐心計數器被持續重置,最終讓 60 節點的數值上限(而非耐心規則本身)結束了搜尋。耐心路徑本身只在一次小預算的驅動腳本煙霧測試中端到端驗證過(`stop_reason` 正確輸出 "2 evals without improvement post-burst");在真實跑動中它會在 burst 是失敗(dud)的比賽上首次被真正檢驗,目前尚無這樣的資料點——這是「auto-stop 有效」這句話目前唯一沒有實戰證據支持的部分,誠實記錄而非假裝已驗證。

5. **F-2 驗證跑暴露的驅動腳本層級 bug(非 harness_v3.py 本身):**s3e7 需要 3 次重啟——(a) blend-vs-solo 分派邏輯錯誤地依字面 lineage 名稱「BLEND」判斷型別而非節點自身的 kind;(b) 模組層級的驅動狀態(lineage 名稱、burst 旗標、node_results)未隨 resume 存活,修正為把這些狀態移進 search_state 本身;(c) 一次 CatBoost 訓練(EXPL_CATDEEP)原生 fit() 卡死 28 分鐘, signal.alarm 無法中斷一個從未返回 Python bytecode 的原生呼叫,只能手動 kill 後重跑(重跑本身只花 54.8 秒)。s3e14 則遇到程序本身的 35 分鐘牆鐘上限在 59/60 節點時觸發、外加 6 次 DART 評估(107–140 秒/次,OOF MAE 高達 6144–6544、約為基準的 18 倍)白白燒掉約 12 分鐘——harness_v3.py 本身在兩場驗證跑中都**零修改**,三個 bug 全部是驅動腳本層級的教訓,見第 13 節的三項上線前工程需求。

13. Stage 4 正式建議

v3 作為預設迴圈

harness_v3.py (相位機/去重耗算/blend 重開/邊界推進/k=800+精修權重搜尋/成本護欄)應作為 Stage 4 樹搜尋的**預設迴圈**,理由:(1) 兩場 F-2 驗證跑都端到端跑通、且雙雙刷新該場全紀錄最佳(s3e7 0.900455、s3e14 340.35572),harness_v3.py 本身在驗證期間零修改;(2) explore burst 機制在全部三次觸發中 3/3 貢獻了 post-exploit-phase 的唯一增益來源(第 10 節);(3) 邊界推進在四個獨立場次確認為單一最大槓桿之一。

上線前必須補齊的三項工程需求(F-2 誠實但書直接指出的缺口,而非鍛造出的新願望清單)

1. **Resume 狀態契約:**驅動腳本自身的執行期狀態(lineage 名稱映射、burst 已注入旗標、node_results 暫存)必須明確定義為 tree["search_state"] 的一部分並隨樹一起持久化,而非留在驅動腳本的模組層級變數——s3e7 F-2 驗證跑的 3 次重啟裡有 1 次直接因此而起,修法已在該跑的驅動腳本裡完成,但尚未提煉成 harness 或驅動腳本模板的正式契約。
2. **子行程層級的評估逾時:** signal.alarm 無法中斷一個從未把控制權交還 Python bytecode 的原生 fit() (s3e7 F-2 一次 CatBoost 訓練卡死 28 分鐘)。每個節點的 solo/blend 評估應該在獨立子行程(subprocess / multiprocessing)裡執行,由父行程強制 kill 逾時的子行程,而非依賴 in-process 訊號。
3. **Burst 種子的健全性閘:**s3e14 F-2 的 6 次 DART 長射程 solo 評估產生的 OOF MAE 高達基準的 18 倍、且每次耗時 107–140 秒(是典型節點的 2–3 倍)——burst 注入的長射程種子需要一個「每種子評估前/評估中」的牆鐘與初步分數健全性檢查(例如:訓練 loss 是否發散、前幾輪 wall 是否遠超過歷史同類節點的中位數),提早中止明顯失控的長射程嘗試,而非讓它跑到底才發現是浪費。

預算規則(Phase E-5 驗證,已編碼為 harness_v3.py 常數,第 11 節)

- 預設總節點預算 **60**(exploit 階段 ~35–40 + 強制 explore burst 5–8 條長射程 lineage)。
- **Explore burst 不可省略**,即使 exploit 階段「看起來已經收斂」——三份獨立證據(E-5/F-2×2)顯示 burst 是 post-exploit 階段唯一的增益來源。
- Burst 後連續 **15–20** 次評估無改善即停(v3 預設 20);若相位機或數值訊號都未乾淨觸發,以 **60** 節點數值上限保底(E-5 驗證:全部 15 次執行的 best/total 比值均值 0.647、介於 0.1–1.0 之間,本次 E-5 自身 0.65——上限留有充分餘裕,幾乎不會截斷真實改進)。

14. 重現指令(v1 + v2 + v3 + 規模實驗,15 次執行)

```
cd /home/tjyen/ai_agents/kaggle

# --- v1(harness.py,Phase C-2a/b/c)---
```

```

uv run python3 tree_search/run_s3e9.py      # 單模型節點空間
uv run python3 tree_search/run_s3e14.py     # solo + blend 節點空間
uv run python3 tree_search/run_s3e5.py      # 離散化指標(QWK-after-rounder)泛化測試

# --- v2 D 掃描(harness_v2.py,Phase D-2..D-6)---
uv run python3 tree_search/run_s3e3.py      # AUC,小樣本(1,677 列)
uv run python3 tree_search/run_s3e7.py      # AUC,42k 列
uv run python3 tree_search/run_s3e1.py      # RMSE,地理特徵,37k 列
uv run python3 tree_search/run_s3e19.py     # SMAPE,TimeSeriesSplit
uv run python3 tree_search/run_s3e11.py     # RMSLE,36 萬列

# --- v2 E 掃描(harness_v2.py,Phase E-1..E-4)---
uv run python3 tree_search/run_s3e9_v2.py   # 復仇戰:ensemble-default 能否翻盤 v1 敗場
uv run python3 tree_search/run_s3e5_v2.py   # 復仇戰:v2 能否破 v1 的平手
uv run python3 tree_search/run_s3e16_v2.py  # 酸性測試:取整 MAE
uv run python3 tree_search/run_s3e20_v2.py  # 結構主宰地形

# --- v2 E-5 規模實驗 ---
uv run python3 tree_search/run_s3e3_scale.py # 80-評估-節點預算曲線

# --- v3 F-2 驗證跑(harness_v3.py)---
uv run python3 tree_search/run_s3e7_v3.py   # 可中斷/續跑
uv run python3 tree_search/run_s3e14_v3.py  # 可中斷/續跑

```

全部 15 支跑腳本皆可中斷/續跑(每個節點寫入後立即以 temp-file + os.replace 原子寫入對應的

experiments_tree*.json)。若要從零開始重跑,先刪除對應的樹狀態檔與 tree_search/cache_<comp>*/ (皆為可重新產生、已 gitignore 的暫存)。

事實抽取與驗證:

```

uv run python3 docs/scripts/build_tree_facts.py
uv run python3 .claude/skills/kaggle-report/assets/verify_report.py docs/tree_search_prototype.md docs
bash .claude/skills/kaggle-report/assets/md2pdf.sh docs/tree_search_prototype.md

```